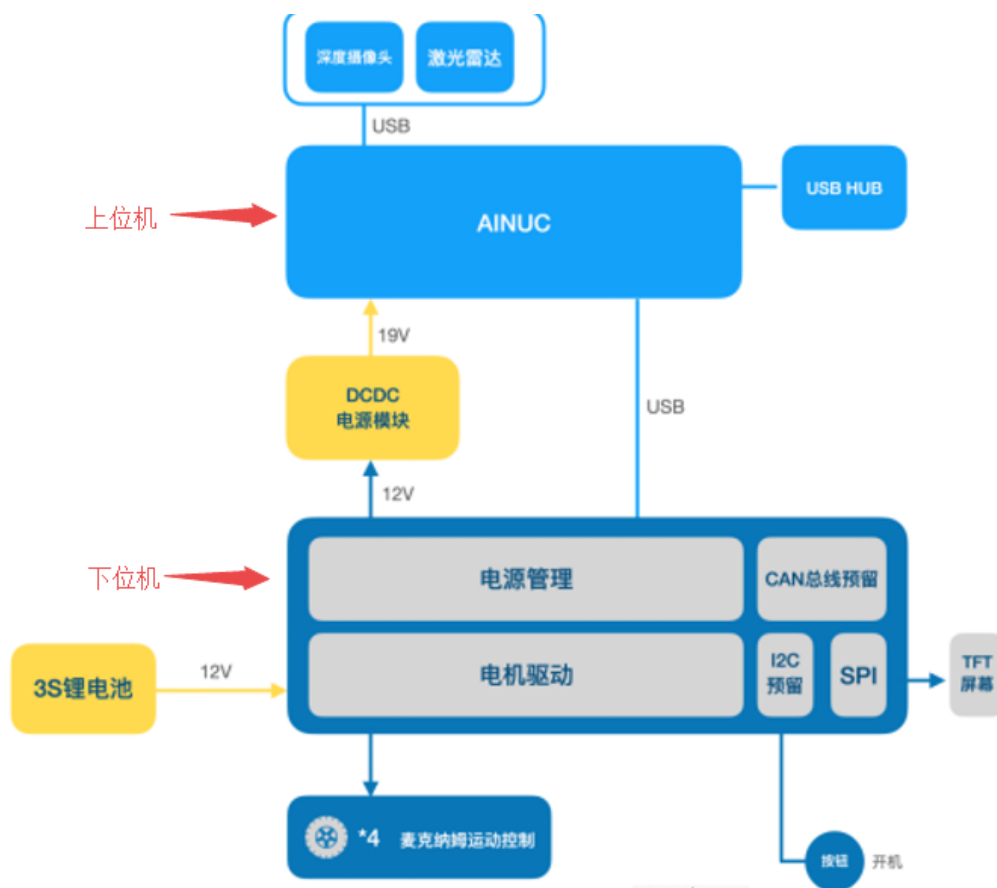


使用串口通信技术实现Arduino与Python的通信

一、任务描述

本次实训课程使用的自动驾驶汽车用英特尔NUC作为上位机，Arduino板作为下位机，上位机直接与下位机进行通信，下位机控制车轮、电池、和屏幕等设备。本次实训课程要求同学实际控制车辆的硬件，来理解上位机、下位机之间的通信。



自动驾驶汽车结构图

二、任务目标

- 使用CuteCom对车轮的信息进行接收和发送，实现不同命令模式的发送，查看小车的反应。
- 掌握Python与串口进行通信的原理。
- 通过实际案例的运行，查看python端（上位机）对车轮和其他硬件的控制。

三、任务环境

- 操作系统：Ubuntu20.04 ROS-neotic
- 工具软件：NoMachine

- 必备硬件：自动驾驶汽车

四、任务原理

4.1 通信基础认知

4.1.1 并行通信

通过输入/输出端口在 Arduino 和外设之间进行并行连接是短距离（最多几米）的理想解决方案。然而，在其他情况下，当需要在两个设备之间建立较长距离的通信时，不可能使用并行连接。并行接口同时传输多个位。它们通常需要数据总线——通过八条，十六条或更多的线路进行传输。数据以1和0的巨大波形传输。

并行通信的优点和缺点：

并行通信虽然可以多位数据同时传输，速度更快，而且控制简单，但其占用的I/O口较多，而Arduino的I/O资源较少，因此在 Arduino 中更常使用的是串行通信方式。

4.1.2 串行通信

串行通信的最重要的事情之一是协议，应该严格遵守。它是一套规则，必须应用这些规则才能使设备正确地解释它们相互交换的数据。但是，Arduino 会自动处理这个问题，这样程序员/用户的工作就可以简化为简单的写（发送的数据）和读（接收的数据）。

- 优点：传输线少，长距离传送时成本低
- 缺点：数据的传送控制比并行通信复杂

串行通信类型：

- 同步 - 同步的设备使用相同的时钟，它们的时序彼此同步。
- 异步 - 异步的设备具有各自的时钟，并由前一状态的输出触发。
如果给所有连接的设备提供相同的时钟，则它们是同步的。如果没有时钟线，它是异步的。

传输方向：

- 单工：是指数据传输仅能沿一个方向，不能实现反向传输。
- 半双工：是指数据传输可以沿两个方向，但需要分时进行。
- 全双工：是指数据可以同时进行双向传输。

4.2 串口通信

串口通信 (Serial Communications) 是指串口按位 (bit) 发送和接收字节。串口用于ASCII码字符的传输，通信使用3根线完成，分别是地线、发送线、接收线。由于串口通信是异步的，端口能够在同一根线上发送数据同时在另一根线上接收数据，其他线用于握手，但不是必须的。串口通信最重要的参数是波特率、数据位、停止位和奇偶校验。对于两个进行通信的端口，这些参数必须要匹配。

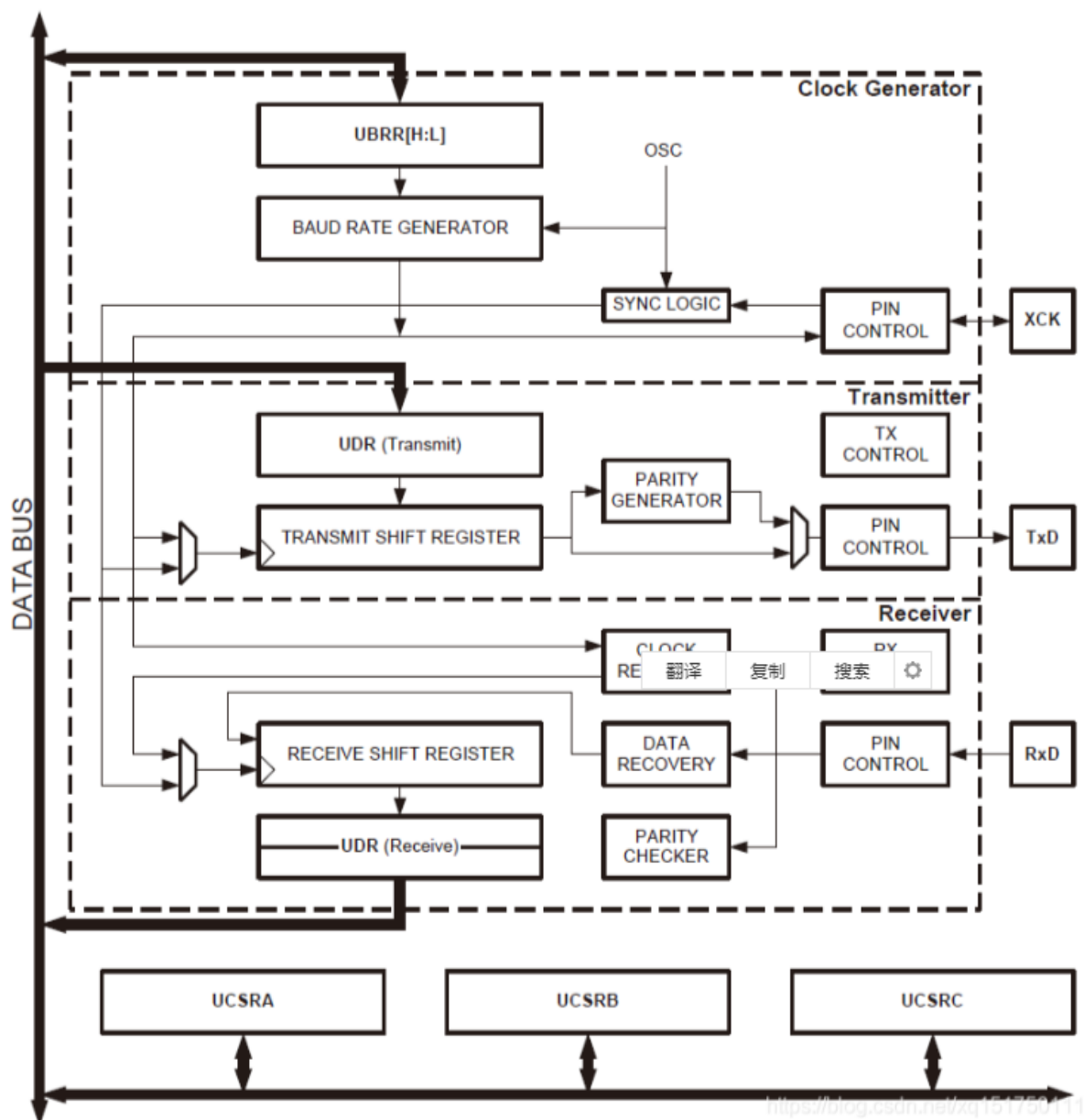
通信速率是指单位时间内传输的信息量，可用比特率和波特率来表示。比特率是指每秒传输的二进制位数，用bps (bits)表示。波特率是指每秒传输的符号数，若每个符号所含的信息量为1比特，则波特率等于比特率。在电子学中，一个符号的含义为高电平或低电平，它们分别代表"1"和"0"，所以，每个符号所含的信息量刚好为1比特，因此常将比特率称为波特率，即：1波特(B) = 1比特(bit) = 1位/秒(1bps)

通信双方需要使用一致的波特率才能正常通信。Arduino串口通信通常会使用以下波特率：300、600、1200、2400、4800、9600、14400、19200、28800、38400、57600、115200等，最常用的是9600。波特率越大，说明串口通信的速率越快。

4.2.1 Arduino串口的硬件结构

单片机是基于微控制器（MCU）搭建的电子系统。单片机的所有功能其实都是由板载的MCU提供的，Arduino开发板当然也不例外。这里以Arduino MEGA 2560为例，它的板载MCU为ATmega2560。

在ATmega2560内部，实现串口的部件为USART（Universal Synchronous and Asynchronous serial Receiver and Transmitter，通用同步/异步串行收发器）。USART内部结构是十分复杂的，主要由三部分组成：时钟发生器、发射器和接收器。每个单元的功能全部由硬件实现，同时以寄存器的形式对用户开放了配置接口（控制寄存器），又以寄存器的形式对用户开放了过程监控（状态寄存器）。



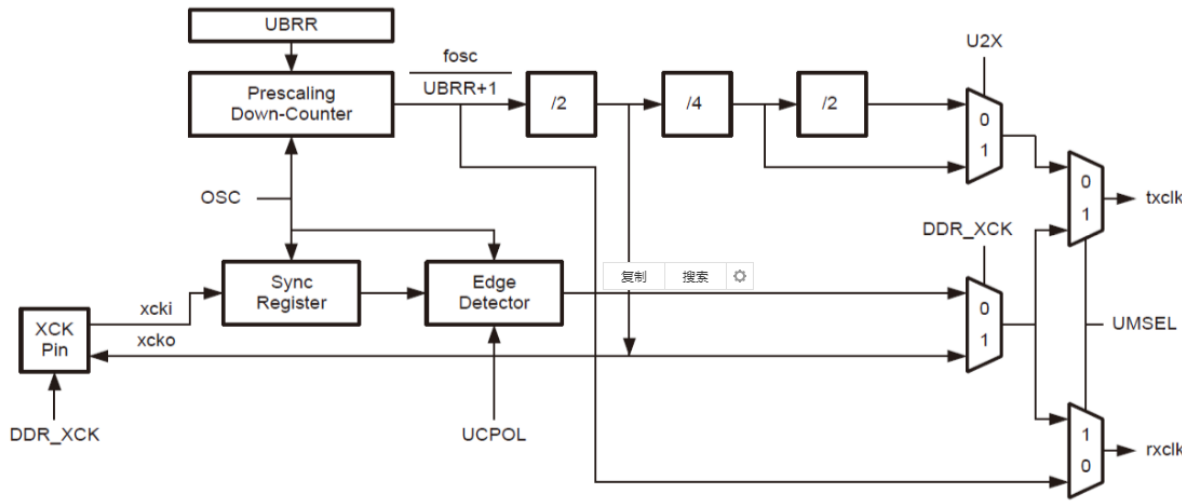
1 USART内部结构

图中的虚线框分隔了USART的三个主要部分(从顶部列出)：时钟发生器、发射器和接收器。控制寄存器由所有单元共享。时钟生成逻辑由同步从操作使用的外部时钟输入的同步逻辑和波特率发生器组成。XCKn(传输时钟)引脚仅用于同步传输模式。发射器由一个写缓冲器、一个串行移位寄存器、奇偶校验发生器和控制逻辑组成，用于处理不同的串行帧格式。写缓冲区允许连续传输数据，帧之间没有任何延迟。由于其时钟和数据恢复单元，接收器是USART模块中最复杂的部分。恢复单元用于异步数据接

收。除了恢复单元，接收器还包括奇偶校验器、控制逻辑、移位寄存器和两级接收缓冲器。接收器支持与发送器相同的帧格式，可以检测帧错误、数据溢出和奇偶校验错误。

时钟发生器为发送器和接收器生成基准时钟。USARTn支持四种时钟操作模式：正常异步、双速异步、主同步和从同步模式。USART控制状态寄存器UCSRnC中的UMSELn位在异步和同步操作之间进行选择。双速(仅异步模式)由UCSRnA寄存器中的U2Xn控制。当使用同步模式(UMSELn = 1)时，XCKn引脚(DDR_XCKn)的数据方向寄存器控制时钟源是内部(主机模式)还是外部(从机模式)。XCKn引脚仅在使用同步模式时有效。

Figure 22-2. Clock Generation Logic, Block Diagram



Signal description:

- txclk** Transmitter clock (Internal Signal).
- rxclk** Receiver base clock (Internal Signal).
- xcki** Input from XCK pin (internal Signal). Used for synchronous slave operation.
- xcko** Clock output to XCK pin (Internal Signal). Used for synchronous master operation.
- fosc** XTAL pin frequency (System Clock).

<https://blog.csdn.net/xq151750111>

2 时钟发生器的内部结构

USART波特率寄存器(UBRRn)和与之相连的递减计数器作为可编程预分频器或波特率发生器。在系统时钟(fosc)下运行的递减计数器，每次计数器递减到零或写入UBRRn寄存器时，都加载UBRRn值。每次计数器归零时都会产生一个时钟。该波特率发生器的时钟输出为 $f_{osc}/(UBRRn+1)$ 。发射器根据模式将波特率发生器时钟输出除以2、8或16。接收器的时钟和数据恢复单元直接使用波特率发生器输出。

Table 22-1. Equations for Calculating Baud Rate Register Setting

Operating Mode	Equation for Calculating Baud Rate ⁽¹⁾	Equation for Calculating UBRR Value
Asynchronous Normal mode (U2Xn = 0)	$BAUD = \frac{f_{osc}}{16(UBRRn + 1)}$	$UBRRn = \frac{f_{osc}}{16BAUD} - 1$
Asynchronous Double Speed mode (U2Xn = 1)	$BAUD = \frac{f_{osc}}{8(UBRRn + 1)}$	$UBRRn = \frac{f_{osc}}{8BAUD} - 1$
Synchronous Master mode	$BAUD = \frac{f_{osc}}{2(UBRRn + 1)}$	$UBRRn = \frac{f_{osc}}{2BAUD} - 1$

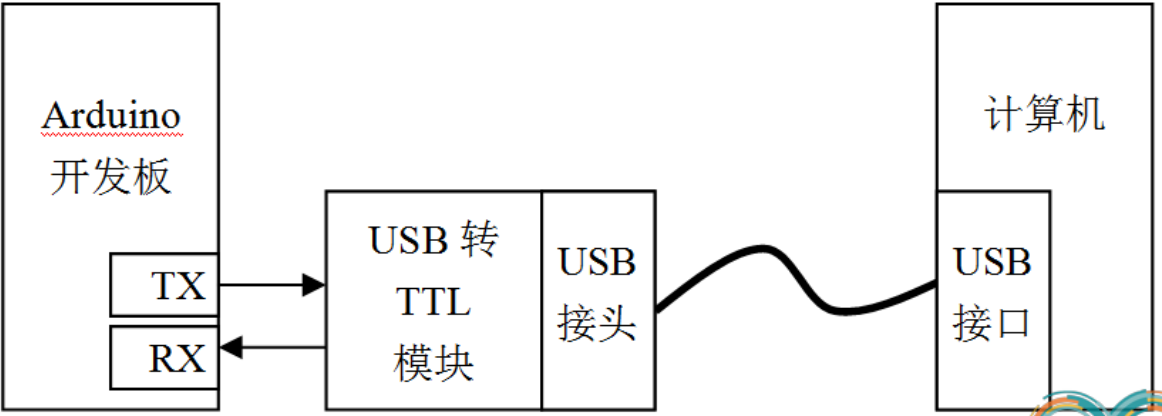
Note: 1. The baud rate is defined to be the transfer rate in bit per second (bps).

- BAUD**
- Baud rate (in bits per second, bps).
- f_{osc}**
- System Oscillator clock frequency.
- UBRRn**
- Contents of the UBRRHn and UBRRLn Registers, (0-4095).

<https://blog.csdn.net/xq151750111>

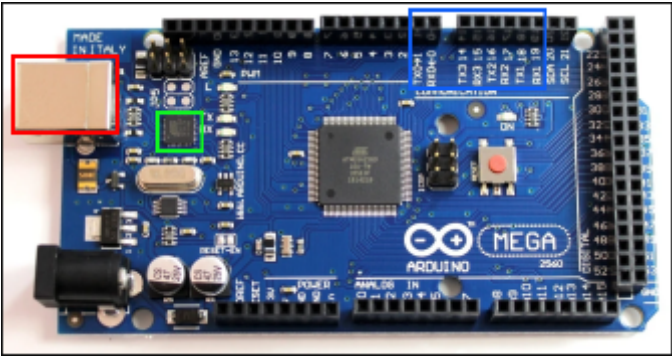
3 UBRRn寄存器值与波特率的对应关系

通过Arduino上的USB接口与计算机连接而进行Arduino与计算机之间的串口通信。除此之外，还可以使用串口引脚连接其他的串口设备进行通信。需要注意的是，通常一个串口只能连接一个设备进行通信。



4 单片机与计算机间的串口通讯方式

常见的USB转TTL芯片有CH340、PL2303、FT232、CP2102等，但现在的Arduino开发板也内置了USB到TTL的转换芯片，官方版本采用Atmega16U2芯片，而国产版本多采用CH340或FT232芯片，因此，可以直接使用USB线连接计算机，实现与上位机的串口通讯了！

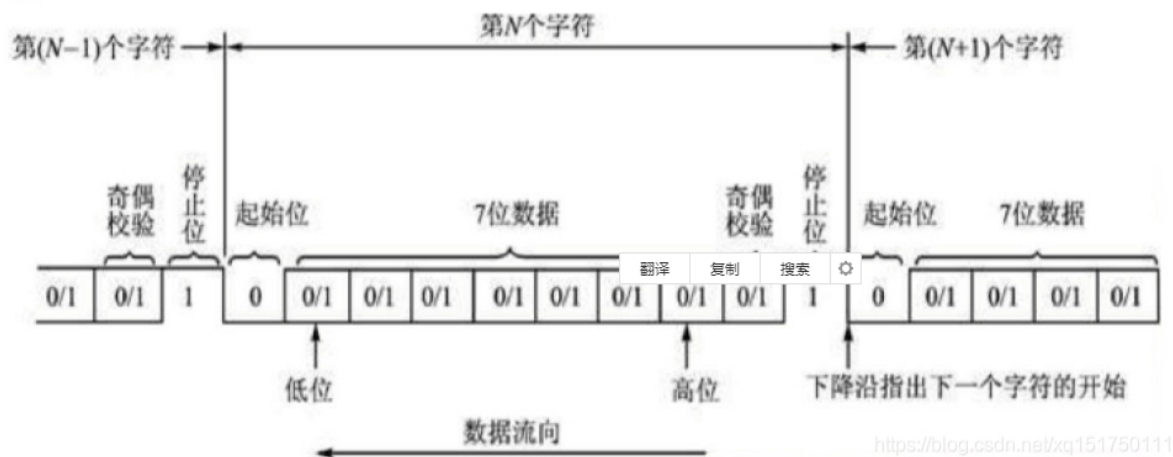


5 Arduino串口的硬件设计

图中蓝色线框的TX、RX引脚为TTL电平，可用于两个单片机开发板之间直接的串口通讯；红色线框内的接头为USB接头，可直接用线与电脑的USB口连接；图中绿色线框内的部分实现了TTL到USB的转换。

4.2.2 串口工作原理

在 Arduino 与其他器件通信的过程中，数据传输实际上都是以数字信号(即电平高低变化)的形式进行的，串口通信也是如此。当使用 `Serial.print()` 函数输出数据时，Arduino 的发送端会输出一连串的数字信号，称这些数字信号为数据帧。



6 串口数据帧格式

(1) 起始位:

起始位总为低电平，是一组数据帧开始传输的信号。

(2) 数据位:

数据位是一个数据包，其中承载了实际发送的数据的数据段。当Arduino通过串口发送一个数据包时，实际的数据可能不是8位的，比如，标准的ASCII码是0-127(7位)。而扩展的ASCII码则是0-255(8位)。如果数据使用简单的文本(标准ASCII码)，那么每个数据包将使用7位数据。Arduino 默认使用8位数据位，即每次可以传输1B数据。

(3) 校验位:

校验位是串口通信中一种简单的检错方式。可以设置为偶校验或者奇校验。当然，没有校验位也可以。Arduino 默认无校验位。

(4) 停止位:

每段数据帧的最后都有停止位表示该段数据帧传输结束。停止位总为高电平，可以设置停止位为1位或2位。Arduino默认是1位停止位。当串口通信速率较高或外部干扰较大时，可能会出现数据丢失的情况。为了保证数据传输的稳定性,最简单的方式就是降低通信波特率或增加停止位和校验位。在Arduino中，可以通过 `Serial.begin(speed,config)` 语句配置串口通信的数据位、停止位和校验位参数。

4.3 Ubuntu下固定硬件设备编号

要让上位机正确识别自动驾驶汽车的设备（如激光雷达、摄像头等），就需要在上位机系统将硬件设备固定，以便与各类硬件进行正确的通信接下来同学们可以跟着以下步骤进行操作。

4.3.1 查看串口信息

用如下命令查看某个串口信息：

```
udevadm info /dev/ttyUSB0
```

```
shawfy@shawfy-pc:~$ udevadm info /dev/ttyUSB0
P: /devices/pci0000:00/0000:00:14.0/usb3/3-3/3-3:1.0/ttyUSB0/tty/ttyUSB0
N: ttyUSB0
S: lidarl
S: serial/by-id/usb-Silicon_Labs_CP2102_USB_to_UART_Bridge_Controller_0001-if00-port0
S: serial/by-path/pci-0000:00:14.0-usb-0:3:1.0-port0
E: DEVLINKS=/dev/serial/by-id/usb-Silicon_Labs_CP2102_USB_to_UART_Bridge_Controller_0001-if00-port0 /dev/lidar /dev/serial/by-path/pci-0000:00:14.0-usb-0:3:1.0-port0
E: DEVNAME=/dev/ttyUSB0
E: DEVPATH=/devices/pci0000:00/0000:00:14.0/usb3/3-3/3-3:1.0/ttyUSB0/tty/ttyUSB0
E: ID_BUS=usb
E: ID_MM_CANDIDATE=1
E: ID_MODEL=CP2102_USB_to_UART_Bridge_Controller
E: ID_MODEL_ENC=CP2102\x20USB\x20to\x20UART\x20Bridge\x20Controller
E: ID_MODEL_FROM_DATABASE=CP210x UART Bridge / myAVR mySmartUSB light
E: ID_MODEL_ID=ea60
E: ID_PATH=pci-0000:00:14.0-usb-0:3:1.0
E: ID_PATH_TAG=pci-0000_00_14_0-usb-0_3_1_0
E: ID_PCI_CLASS_FROM_DATABASE=Serial bus controller
E: ID_PCI_INTERFACE_FROM_DATABASE=XHCI
E: ID_PCI_SUBCLASS_FROM_DATABASE=USB controller
E: ID_REVISION=0100
E: ID_SERIAL=Silicon_Labs_CP2102_USB_to_UART_Bridge_Controller_0001
E: ID_SERIAL_SHORT=0001
E: ID_TYPE=generic
E: ID_USB_DRIVER=cp210x
E: ID_USB_INTERFACES=:ff0000:
E: ID_USB_INTERFACE_NUM=00
E: ID_VENDOR=Silicon_Labs
E: ID_VENDOR_ENC=Silicon\x20Labs
E: ID_VENDOR_FROM_DATABASE=Cygnal Integrated Products, Inc.
E: ID_VENDOR_ID=10c4
E: MAJOR=188
E: MINOR=0
```

上图的“3-3:1.0”代表电脑上这个USB口编号，也就是说只要连在这个USB端口上的串口都叫这个ID。

4.3.2 创建配置文件

使用如下命令创建配置文件：

```
vi /etc/udev/rules.d/com_port.rules
```

内容示例如下：

```
(base) mo@mo-AI:~$ cat /etc/udev/rules.d/com_port.rules
KERNEL=="ttyUSB*", ATTRS{idVendor}=="1a86", ATTRS{idProduct}=="7523", MODE:="0666", SYMLINK+="car_base"
KERNEL=="ttyUSB*", ATTRS{idVendor}=="0403", ATTRS{idProduct}=="6001", MODE:="0666", SYMLINK+="car_base"
KERNEL=="ttyACM*", ATTRS{idVendor}=="2341", ATTRS{idProduct}=="0042", MODE:="0666", SYMLINK+="car_base"

KERNEL=="ttyUSB*", ATTRS{idVendor}=="10c4", ATTRS{idProduct}=="ea60", MODE:="0666", SYMLINK+="RP_lidar"

KERNEL=="video*", ATTRS{idVendor}=="2bc5", ATTRS{idProduct}=="0511", ATTR{index}=="0", MODE:="0666", SYMLINK+="video10"
KERNEL=="video*", ATTRS{idVendor}=="0801", ATTRS{idProduct}=="0101", ATTR{index}=="0", MODE:="0666", SYMLINK+="video10"
KERNEL=="video*", ATTRS{idVendor}=="1bcf", ATTRS{idProduct}=="2cc8", ATTR{index}=="0", MODE:="0666", SYMLINK+="video10"
SUBSYSTEM=="usb", ATTR{idVendor}=="2bc5", ATTR{idProduct}=="0614", MODE:="0666", OWNER:="root", GROUP:="video", SYMLINK+="astradept"
```

- KERNEL 内核设备的名称，例如 ttyUSB*，sda，等，是处于/dev 目录下的设备名称。
- idVendor：每台设备都有一个idVendor，厂家标识。
- idProduct：产品标示。其中厂家标示是供应商ID，是唯一的，产品标示则是厂家自定义的ID。通过 lsusb 可以查看USB的设备信息。
- MODE：赋予设备权限。采用 0777 读写权限即可。
- SYMLINK：SYMLINK为 /dev/下的设备文件产生符号链接。因为 udev 只能为某个设备产生一个设备文件，所以为了不覆盖系统默认的 udev 规则产生的设备文件，推荐使用符号链接。

4.3.3 绑定USB串口号

使用lsusb查看设备：

```
(base) mo@mo-AI:/etc/udev/rules.d$ lsusb
Bus 002 Device 001: ID 1d6b:0003 Linux Foundation 3.0 root hub
Bus 001 Device 008: ID 10c4:ea60 Silicon Labs CP210x UART Bridge
Bus 001 Device 006: ID 045e:028e Microsoft Corp. Xbox360 Controller
Bus 001 Device 009: ID 2bc5:0614
Bus 001 Device 007: ID 2bc5:0511 USB Serial
Bus 001 Device 005: ID 05e3:0610 Genesys Logic, Inc. 4-port hub
Bus 001 Device 003: ID 05e3:0608 Genesys Logic, Inc. Hub
Bus 001 Device 004: ID 8087:0aaa Intel Corp.
Bus 001 Device 002: ID 1a86:7523 QinHeng Electronics HL-340 USB-Serial adapter
Bus 001 Device 001: ID 1d6b:0002 Linux Foundation 2.0 root hub
```

列出所有的usb设备，将需要固定串口号的设备添加到/etc/udev/rules.d/com_port.rules配置文件中。例如，倒数第二行ID为1a86:7523的设备（已经绑定好了）对应4.3.2图中的第一行。

使用下面命令查看绑定结果：

```
ll /dev | grep ttyUSB
```

```
(base) mo@mo-AI:/etc/udev/rules.d$ ll /dev | grep ttyUSB
lrwxrwxrwx 1 root root 7 3月 16 15:56 car_base -> ttyUSB0
lrwxrwxrwx 1 root root 7 3月 16 15:56 RP_lidar -> ttyUSB1
crw-rw-rw- 1 root dialout 188, 0 3月 16 16:26 ttyUSB0
crw-rw-rw- 1 root dialout 188, 1 3月 16 15:56 ttyUSB1
```

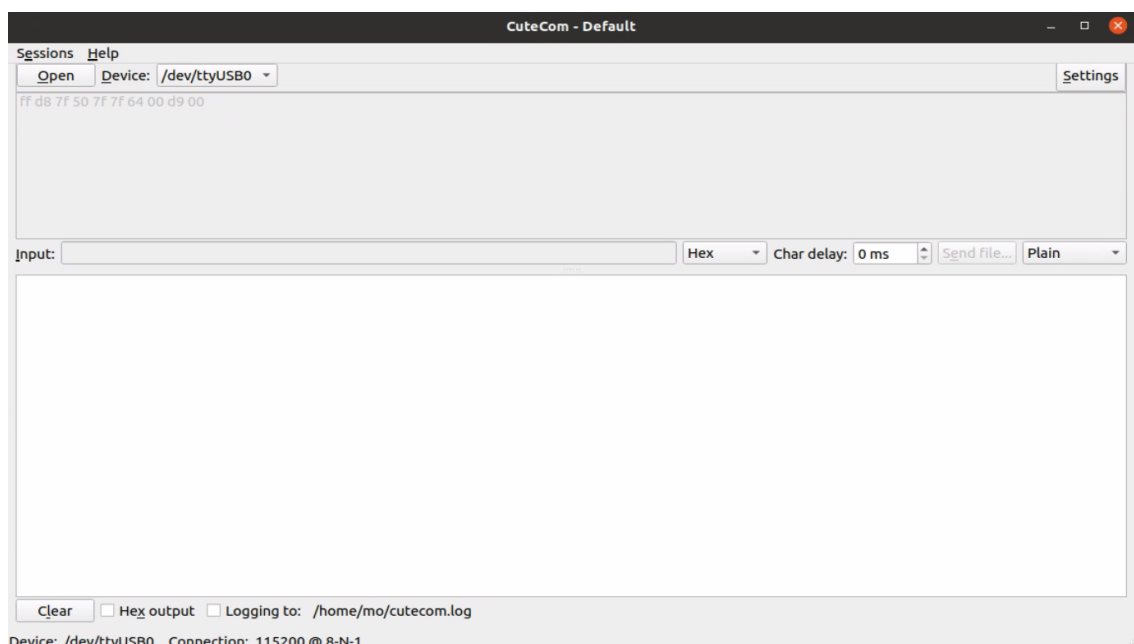
五、任务步骤

步骤一 使用CuteCom实现串口通信的可视化

通过上位机安装的CuteCom，对车轮信息进行接受和发送：

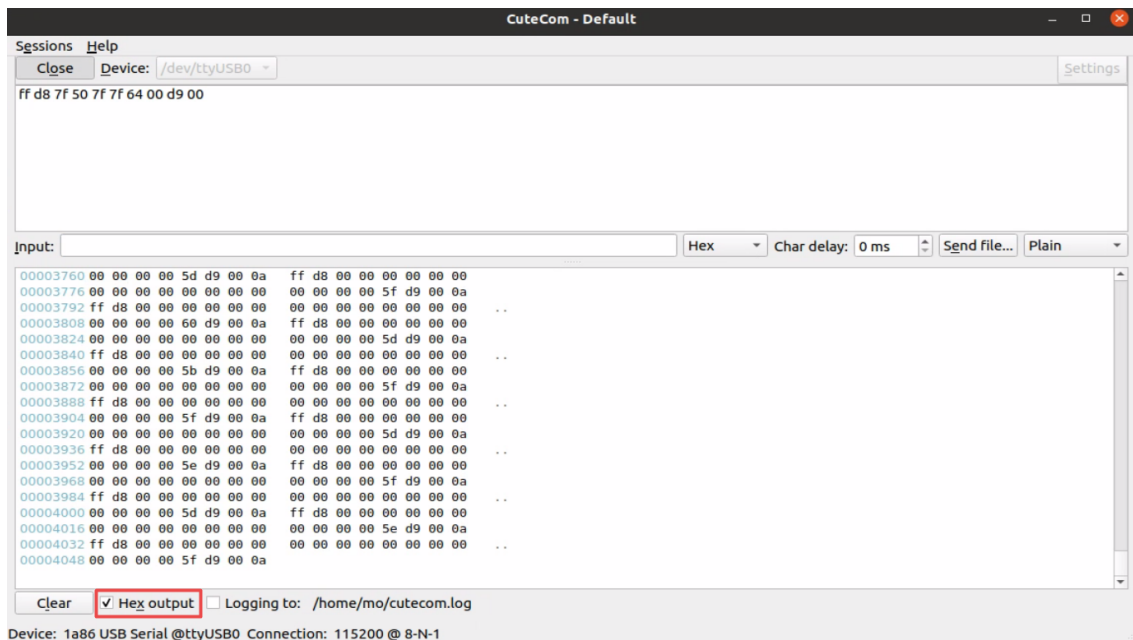
1.1 接收车轮信息数据

1. 在上位机中打开“CuteCom”：

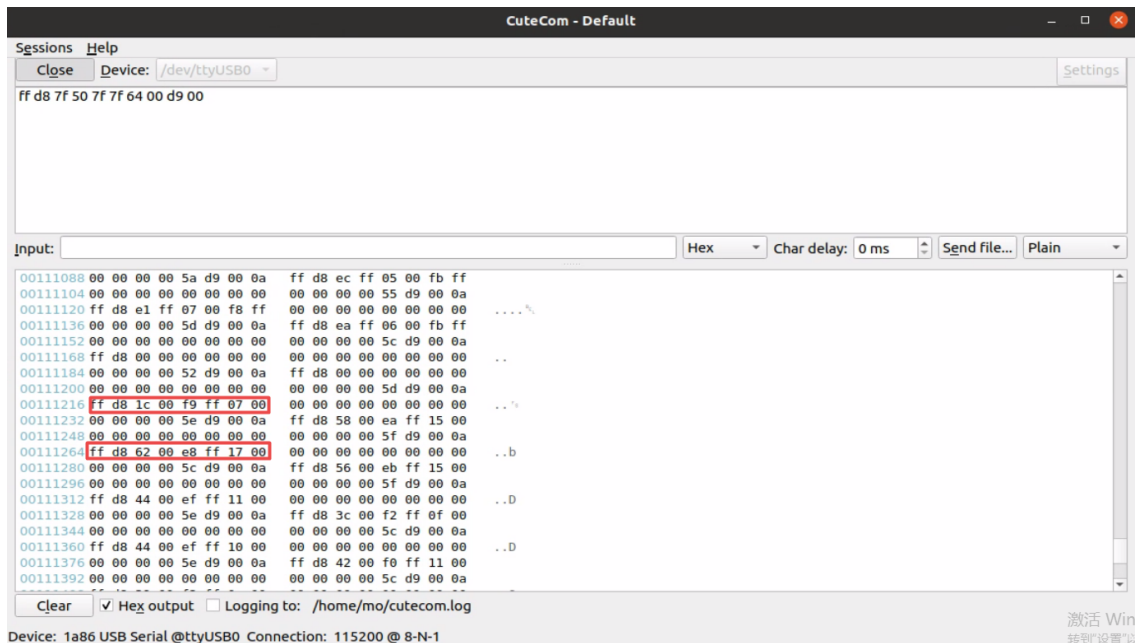


2. 打开端口接收车轮数据：

的“Device”选项选择“/dev/ttyUSB0”，点击“Open”：



选择左下角“Hex output”，可以看到不断闪烁更新的数据以16进制方式显示。当车轮不转动时，我们可以发现，返回的数据几乎不会发生变化。接下来我们手动转动自动驾驶汽车车轮，查看结果：



我们可以发现，转动车轮时，反馈的数据与上图显示有很大差异。

串口接收到的数据格式：

```
//帧头
Serial.write(0xff);
Serial.write(0xd8);

//编码器角度预估（低位、高位）
Serial.write(encoder_angle & 0xFF);
Serial.write((encoder_angle >> 8) & 0xFF);
//编码器x速度（低位、高位）
Serial.write(encoder_vel & 0xFF);
Serial.write((encoder_vel >> 8) & 0xFF);
//编码器y速度（低位、高位）
Serial.write(encoder_y & 0xFF);
Serial.write((encoder_y >> 8) & 0xFF);
```

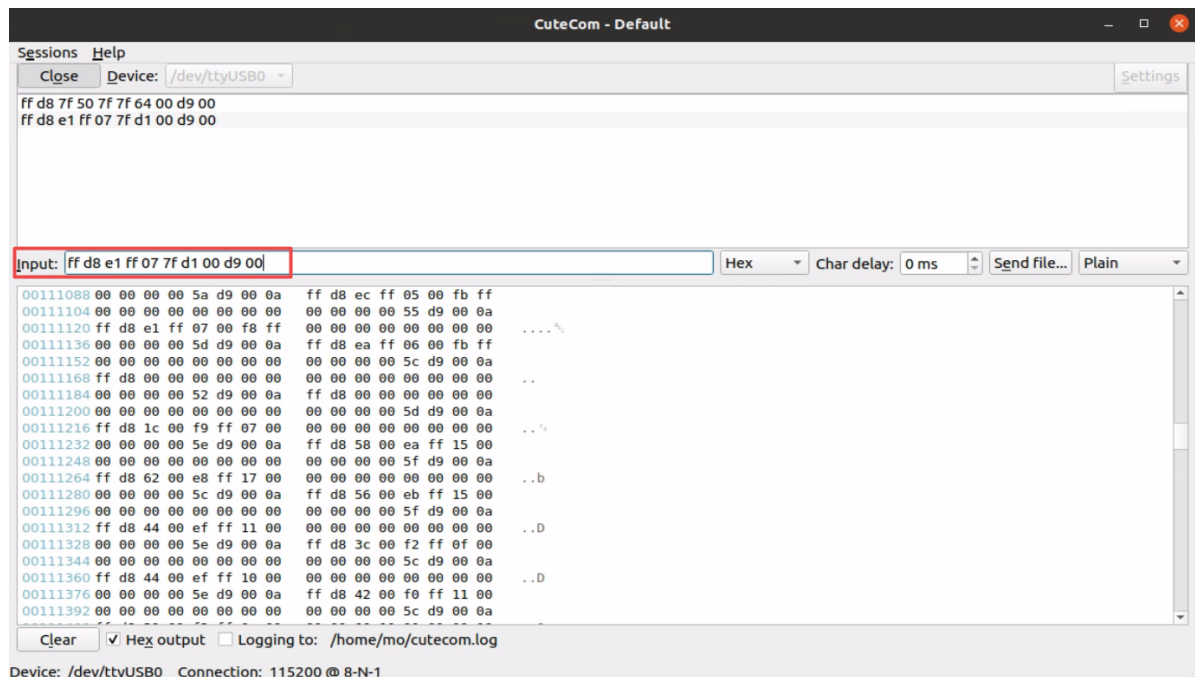
```

//IMU（低位、高位）
//加速度
Serial.write(acc_x & 0xFF);
Serial.write((acc_x >> 8) & 0xFF);
Serial.write(acc_y & 0xFF);
Serial.write((acc_y >> 8) & 0xFF);
Serial.write(acc_z & 0xFF);
Serial.write((acc_z >> 8) & 0xFF);
//角速度
Serial.write(rot_x & 0xFF);
Serial.write((rot_x >> 8) & 0xFF);
Serial.write(rot_y & 0xFF);
Serial.write((rot_y >> 8) & 0xFF);
Serial.write(rot_z & 0xFF);
Serial.write((rot_z >> 8) & 0xFF);
//电池数据
Serial.write(battery_value);
//帧尾
Serial.write(0xd9);
Serial.write(0x00);
Serial.write('\n')

```

1.2 发送数据，控制车轮运动：

在“CuteCom”的“Input”中输入：“ff d8 e1 ff 07 7f d1 00 d 00”，回车发送，可以看到车轮运动：



CuteCom发送数据的格式：

```

# 帧头
send_list.append(0xff)
send_list.append(0xd8)

send_list.append(angle_)      # 旋转角度/第1位ip地址数字
send_list.append(throttle_)  # 前后速度/第2位ip地址数字
send_list.append(rampage_)    # 左右平移速度/第3位ip地址数字

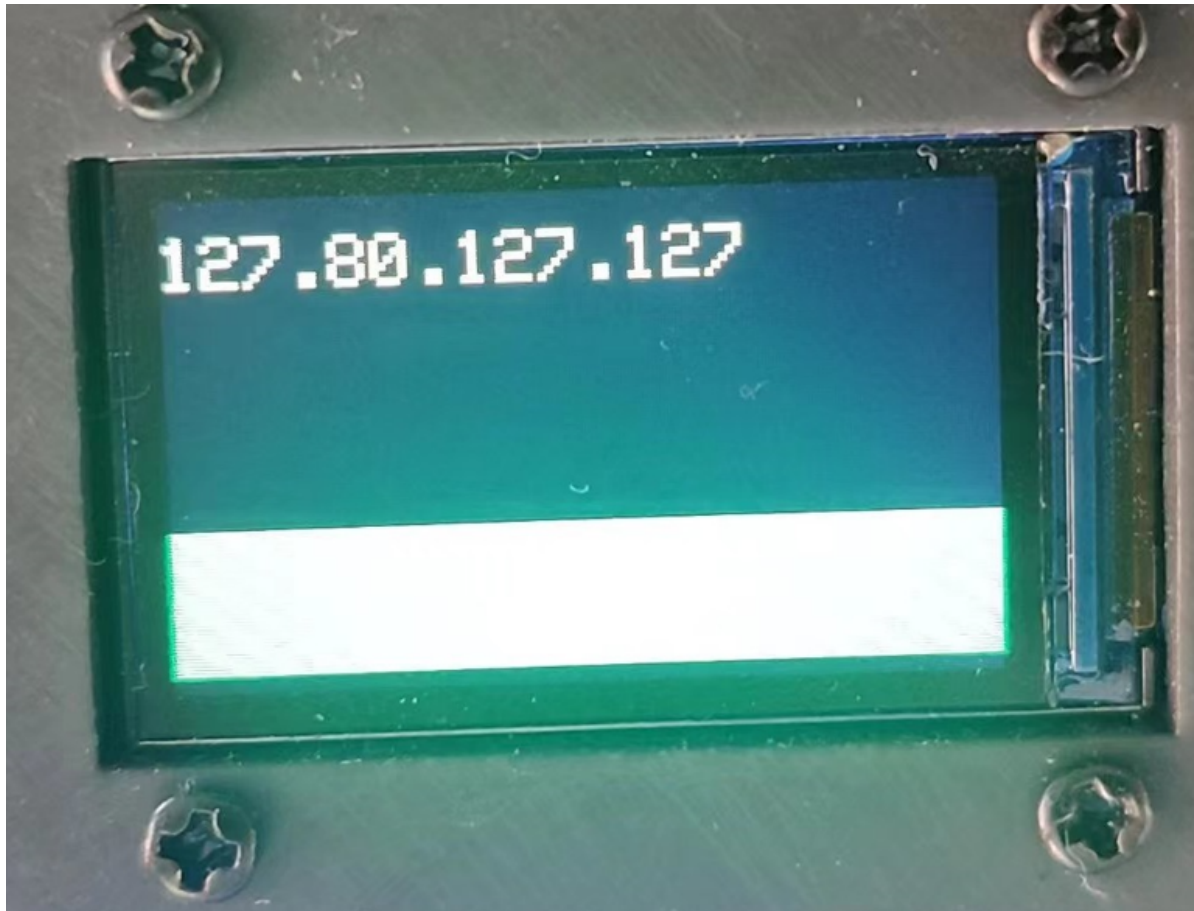
```

```
send_list.append(0x7f)      # 第4位ip地址数字（地址发送模式）
send_list.append(battery)   # 电池信息
send_list.append(0x00)      # cmd, 控制底盘模式

# 帧尾
send_list.append(0xd9)
send_list.append(0x00)
```

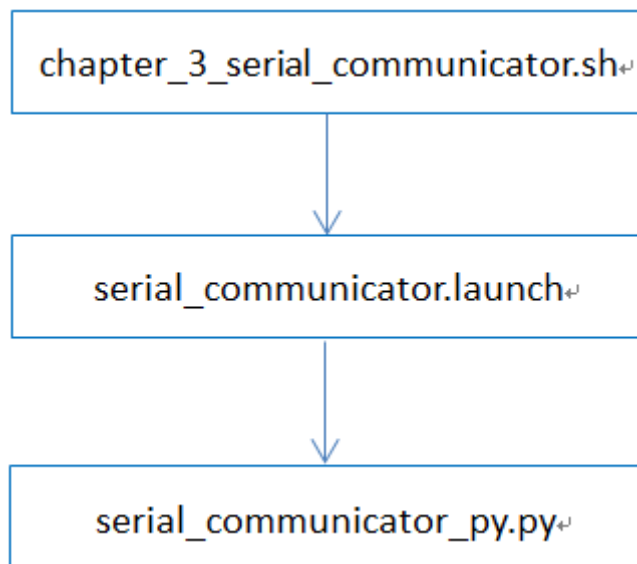
- cmd字段是模式控制字段: 0x00=麦克纳姆轮控制, 0x01=ip地址发送

在“CuteCom”的“Input”中输入: “ff d8 e1 ff 07 7f d1 01 d 00”, 回车发送, 可以看到自动驾驶汽车屏幕上显示发送的ip地址:



步骤二 python端发送与接收串口数据的实现

此步骤同学可以利用demo案例进行操作, 主要流程示意如下:



首先，通过“/home/mo/ros_learning_demos/sample_shell/chapter_3_serial_communicator.sh”，启动“serial_communicator.launch”

```

chapter_3_serial_communicator.sh  serial_communicator.launch
1 #!/bin/bash
2 # source /opt/ros/noetic/setup.bash
3 # source ~/ros_space/devel/setup.bash
4
5 gnome-terminal "启动串口节点" -- bash -c " roslaunch learning_sensor serial_communicator.launch killall roslaunch"
  
```

再通

过“/home/mo/ros_learning_demos/src/learning_ros/learning_sensor/launch/serial_communicator.launch”文件执行“serial_communicator_py.py”脚本：

```

chapter_3_serial_communicator.sh  serial_communicator.launch
1 launch
2 <!-- 底盘demo节点 -->
3 <node pkg="learning_sensor" type="serial_communicator_py.py" name="serial_communicator" output="screen" required="true">
4   </node>
5
6 </launch>
7
  
```

最后在“/home/mo/ros_learning_demos/src/learning_ros/learning_sensor/scripts/”目录中创建脚本“serial_communicator_py.py”，输入如下内容：

```

#!/usr/bin/python3

import struct
import serial,time,os,rospy

battery = 0
car_vel = [0]*5
data = [0]*9

# 判断串口是否存在

def checkTTY():
    USB0_if_exist = os.path.exists("/dev/car_base")
    if (USB0_if_exist):
        dev_fn = '/dev/car_base'
        time.sleep(0.6)
        return USB0_if_exist, dev_fn

# 串口写入方法

def carVelwirte():
  
```

```

global ser
global battery
angle_ = round((car_vel[0] +1.0) * 126.5 + 1.0)
throttle_ = round((car_vel[1] +1.0) * 126.5 + 1.0)
rampage_ = round((car_vel[2] +1.0) * 126.5 + 1.0)
send_list = []
send_list.append(0xff)
send_list.append(0xd8)
send_list.append(angle_)
send_list.append(throttle_)
send_list.append(rampage_)
send_list.append(0x7f)
send_list.append(battery)
send_list.append(0x00)
send_list.append(0xd9)
send_list.append(0x00)
ser.write(serial.to_bytes(send_list))

def scalarization(speed):
    if speed > 0.5:
        speed = 0.5
    if speed < -0.5:
        speed = -0.5
    return speed

# 主函数

if __name__ == '__main__':

    rospy.init_node('serial_communicator_py',anonymous=True)
    USB0_if_exist, dev_fn = checkTTY()

    if(USB0_if_exist):
        ser = None

        # 打开串口

        try:
            ser = serial.Serial(dev_fn, 115200, timeout=1)
        except Exception as e:
            print(e)
            time.sleep(1)

        if(ser is not None and not ser.isOpen()):
            ser.open()

        # 读取键盘输入数据，添加负号为了与ROS中的数据相同

        print('enter rotation:"z", forward speed:"x", lateral speed:"y".')
        car_vel[0] = scalarization(float(input('z:')))
        car_vel[1] = -scalarization(float(input('x:')))
        car_vel[2] = -scalarization(float(input('y:')))
        carVelwirte()

        # 清除串口之前排队中的消息

```

```

ser.read_all()

# 等待3秒后, 返回结果, 并结束循环

temp_time = time.time()
battery_count = 0
while time.time() - temp_time < 3:

    # 判断串口等待读取的字符数量

    time.sleep(0.1)
    res = ser.inWaiting()

    # 因为每次的数据量是24位, 所以每次读取24位

    while not res < 24:
        msg = ser.read(24)
        res = res - 24

    # 判断消息头尾是否正确

    if msg[:2] == b'\xff\xd8' and msg[-3:] == b'\xd9\x00\n':

        # 从3~20位是传感器数据, 使用for循环遍历数据并处理

        for i in range(9):
            temp_data = struct.unpack('<h', msg[2+2*i:4+2*i])
            data[i] += temp_data[0]

        # 第21位是电池电量数据, 计算其平均值

        battery = (battery*battery_count +
int(msg[20]))/(battery_count+1)
        battery_count += 1

    # 打印结果并结束循环, 添加负号为了与ROS中的数据相同

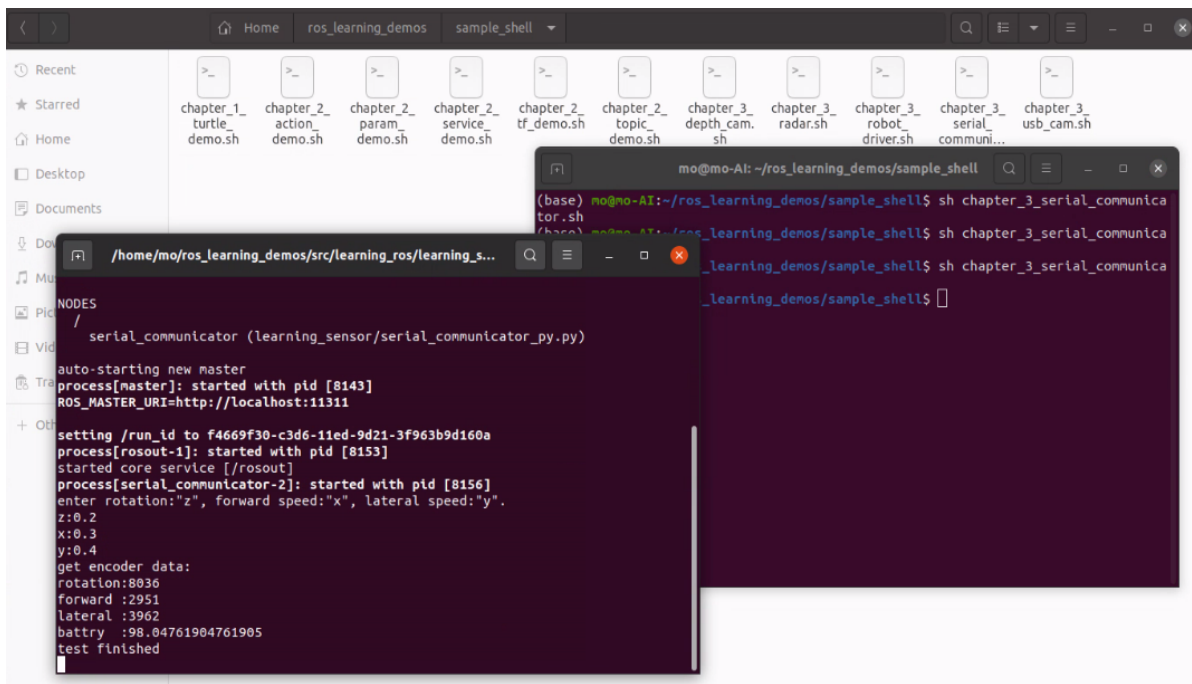
    print('get encoder data:')
    print('rotation: '+str(-data[0]))
    print('forward :'+str(-data[1]))
    print('lateral :'+str(data[2]))
    print('battery :'+str(battery))
    print('test finished')
    time.sleep(5)

else:
    print('没有找到任何串口')

```

实验效果:

运行“/home/mo/ros_learning_demos/sample_shell/chapter_3_serial_communicator.sh”脚本, 输入ZXY值分别为0.2, 0.3, 0.4, 小车发生移动。



ZXY分别为输入的转向、前进、横向移动速度。输出rotation为转向速度，forward为前进速度，lateral为横向移动速度。battery为电池电量信息。

六、任务总结

本次实训任务通过使用不同方式对上位机和下位机之间的通信进行控制，接收并发送信息，达到控制车辆运动的目的。从而深刻理解python和Arduino的通信原理和方式。同学可以反复实验，发送不同的命令数据，查看车辆运动方式。